

PRISM: Privacy-Preserving Smart Contract Auditing with Edge-Aware Graph Attention, Local LLMs, and EVM Validation

Anonymous Author(s)

Anonymous Institution

Abstract. Audits of Ethereum Virtual Machine (EVM) smart contracts increasingly rely on large language models (LLMs), but sending unreleased contract source code to cloud-hosted models exposes proprietary business logic, while rule-based analyzers flood auditors with alarms and fuzzers struggle to reach deep states without guidance. We present **PRISM**, an audit pipeline that runs entirely on local infrastructure and combines (i) reversible identifier pseudonymization (*DataMasker*), (ii) a function-level code property graph (CPG) built from Slither’s static single assignment (SSA) intermediate representation (IR), (iii) an edge-aware multi-head graph attention classifier (*GNNMHA_{v2}*) that ranks functions for (iv) an open-weight quantized LLM that writes Foundry invariant harnesses, and (v) EVM execution and template-based patch checking. On 12,991 function graphs covering five Smart Contract Weakness Classification (SWC) classes, GNNMHA_{v2} reaches 74.09% accuracy and 75.77% macro-F1 on a held-out test set of 1,949 graphs, and 75.75% macro-F1 when the 28 test contract families share no family with training. On a 57-contract benchmark, DataMasker keeps every contract compilable and every Slither finding reproducible, while restricting Slither output to the target taxonomy and to state-mutating findings lowers the share of safe contracts with any alarm from 27/27 to 6/27. We also quantify what pseudonymization does *not* hide: 89.5% of identifiers remain linkable through token-frequency analysis, which is why PRISM keeps all inference local.

Keywords: Smart contract security · Graph neural networks · Large language models · Invariant fuzzing · Privacy-preserving analysis.

1 Introduction

Smart contracts on the Ethereum Virtual Machine (EVM) hold large amounts of value in decentralized finance, and deployed bytecode cannot be patched in place [2, 18]. Incidents from The DAO to repeated flash-loan manipulations show that flaws must be found before deployment [2, 13].

Automated auditing has relied on two paradigms [1, 13, 34]. Static analyzers such as Slither [7] and Securify [27] match intermediate representations (IRs) against expert rules; they are fast but imprecise. In a study of 47,587 contracts,

97% were flagged as vulnerable by at least one tool, indicating a large number of false positives [6], and bug-injection studies confirm both missed bugs and spurious alarms [10]. Dynamic fuzzers [4, 11, 12, 22, 25, 31] produce concrete execution witnesses but must satisfy narrow guards to reach vulnerable states.

Graph neural network (GNN) detectors [17, 20, 23, 33, 35] and auditors built on large language models (LLMs) [9, 15, 16, 26, 30] close part of this gap. Most LLM-based systems, however, call hosted models such as GPT-4 [16, 26], which means transmitting unreleased source code to a third party. Confidential inference on the provider side is an active research topic [14] but is not yet available to typical audit teams. Open-weight models fine-tuned for vulnerability detection [19] show that local inference is feasible, but they classify code rather than produce executable evidence, and they do not analyze what a local pipeline still leaks when its artifacts (prompts, logs, harnesses) are shared.

PRISM addresses this setting. It is a pipeline that never sends code off the audit host and that uses each component for what it does well: a graph classifier to rank functions cheaply, an open-weight LLM to write targeted invariant harnesses for the ranked functions, and the EVM to confirm or reject each hypothesis. Our contributions are:

- **A local audit pipeline** (Sect. 3) that chains reversible pseudonymization, code property graph (CPG) extraction, graph-based screening, local LLM harness generation, Foundry execution, and patch checking, with an explicit threat model (Sect. 3.1).
- **GNNMHAv2**, an edge-aware graph attention classifier over a function-level CPG with typed control- and data-flow edges, jumping-knowledge (JK) readout, and focal loss. It achieves $76.31\% \pm 0.06\%$ macro-F1 across five seeds on a strict lineage-disjoint split and provides 0.21 ms function ranking latency before dynamic execution (Sect. 5.1).
- **A fidelity and exposure analysis of pseudonymization**: DataMasker preserves compilability (57/57) and Slither findings (229/229), and we measure its residual exposure to dictionary (30.4%) and token-frequency (89.5%) linkage attacks (Sect. 5.3).
- **An EVM-level validation study** of harness execution (300 runs, 30 seeds), alarm filtering on 57 contracts reducing false alarms on safe contracts to 0%, and five-check validation of template patches (Sects. 5.2–5.5).

The remainder of this paper is organized as follows. Sect. 2 reviews related work, Sect. 3 presents the threat model and design, Sect. 4 describes the experimental setup, Sect. 5 answers five research questions, Sect. 6 discusses the findings, and Sect. 7 concludes the paper.

2 Related Work

Static analysis and verification. Oyente [18] applies symbolic execution with satisfiability modulo theories (SMT) solving; Securify [27] checks compliance and violation patterns in Datalog; Slither [7] lifts Solidity to a static single assignment

(SSA) IR, SlithIR, and runs rule-based detectors. Empirical studies report low precision and uneven recall across these tools [6, 10].

Fuzzing. ContractFuzzer [12] monitors call-level oracles, Echidna [11] and sFuzz [22] perform property-based and adaptive fuzzing, Harvey [31] and Smartian [4] add greybox and data-flow guidance, and ItyFuzz [25] fuzzes from state snapshots.

Graph learning. Zhuang et al. [35] and Liu et al. [17] learn over contract graphs with normalized nodes and expert patterns; DA-GNN [33] applies dual attention; ConSense [23] combines local and global attention; MANDO-LLM [20] feeds LLM code features into heterogeneous graph transformers. PRISM’s classifier builds on GAT/GATv2 [3, 28] and differs in operating on a homogeneous SSA-level function graph with typed edge attributes. Its role is to rank functions for the LLM stage, not to be the final verdict.

LLMs for auditing, fuzzing, and repair. GPTScan [26] pairs GPT with static confirmation; PropertyGPT [16] and SmartInv [29] generate properties or invariants for verification; LLM4Fuzz [24], LLAMA [9] and PromFuzz [15] use LLMs to guide fuzzing; LLM-SmartAudit [30] uses multi-agent conversation. EVuLLM [19] fine-tunes small open-weight models with QLoRA. On repair, sGuard [21] and Elysium [8] patch source or bytecode with templates, and LLM-based program repair is surveyed in [32]. Table 1 positions PRISM: it is not the strongest detector or fuzzer in isolation. Its contribution is combining ranking, local harness generation and execution under an explicit confidentiality constraint.

3 PRISM Design

Figure 1 shows the five stages. All stages run on the audit host; no network access is required after the model weights are installed.

3.1 Threat Model and Pseudonymization

Assets and trust boundary. The asset is unreleased contract source code together with the business logic it reveals (identifier names, constants, addresses, endpoints). The trust boundary is the audit host operated by the contract owner or its auditor.

Adversaries. We consider (A1) third-party model providers and network observers, which a cloud-LLM workflow would expose code to; and (A2) honest-but-curious parties that later see PRISM *artifacts* outside the host, such as prompt logs, generated harnesses, bug reports shared with external reviewers, or a shared inference server inside the organization. PRISM removes A1 by construction (no remote inference). DataMasker reduces what A2 learns from artifacts. A fully

Table 1. Capability comparison with representative systems, ordered by year. GNN: learned graph detector; LLM: uses an LLM; Local: open-weight model run on premises; Privacy: protects identifiers or source before model input; Execution: confirms findings by execution (F: formal proof instead); Repair: synthesizes patches; n/a: no model is used; NR: not reported.

System	Year	GNN	LLM	Local	Privacy	Execution	Repair
Slither [7]	2019	✗	✗	n/a	n/a	✗	✗
sGuard [21]	2021	✗	✗	n/a	n/a	✗	✓
Smartian [4]	2021	✗	✗	n/a	n/a	✓	✗
Elysium [8]	2022	✗	✗	n/a	n/a	✗	✓
ItyFuzz [25]	2023	✗	✗	n/a	n/a	✓	✗
GPTScan [26]	2024	✗	✓	✗	✗	✗	✗
EVuLLM [19]	2025	✗	✓	✓	✗	✗	✗
MANDO-LLM [20]	2025	✓	✓	NR	✗	✗	✗
PromFuzz [15]	2025	✗	✓	NR	✗	✓	✗
PropertyGPT [16]	2025	✗	✓	✗	✗	F	✗
LLAMA [9]	2026	✗	✓	NR	✗	✓	✗
This work (PRISM)	2026	✓	✓	✓	✓	✓	✓

compromised audit host is out of scope, because such an adversary can read the plaintext source directly.

Mechanism. *PrivacyFilter* first removes hard-coded keys, remote procedure call (RPC) endpoints, and 20-byte addresses. *DataMasker* then renames user-defined identifiers while preserving Solidity keywords, EVM built-ins such as `msg.sender` and `block.timestamp`, and standard interfaces (e.g., `IERC20`). Each identifier x_i of syntactic kind $\kappa(x_i) \in \{\text{type}, \text{function}, \text{variable}\}$ is mapped deterministically to

$$\mathcal{T}(x_i) = \text{prism_}\kappa(x_i)_ \langle i \rangle, \quad (1)$$

where $\langle i \rangle$ is a zero-padded counter. The inverse table $\mathcal{T}^{-1}$ is kept only in process memory and is used to restore names in the final report. Sect. 5.3 measures both fidelity and the residual linkage risk against A2.

3.2 Function-Level Code Property Graph

From the pseudonymized source S_{mask} , PRISM extracts Slither’s SSA IR [7] and builds one directed graph per function, $\mathcal{G} = (\mathcal{V}, \mathcal{E}, X, E_{attr})$, whose nodes are SSA instructions.

- **Node features** $X \in \mathbb{R}^{N \times 32}$: a 13-dimensional one-hot instruction type, 10 security indicators (low-level calls, storage reads and writes, guards, transaction-context reads, etc.), and 9 structural attributes (in- and out-degree, loop membership, relative depth, etc.).
- **Edge attributes** $E_{attr} \in \{0, 1\}^{|\mathcal{E}| \times 4}$: a one-hot edge type among `sequential`, `branch`, `loop_back` (control flow) and `data_dep` (def-use), so control and data flow share one edge set.

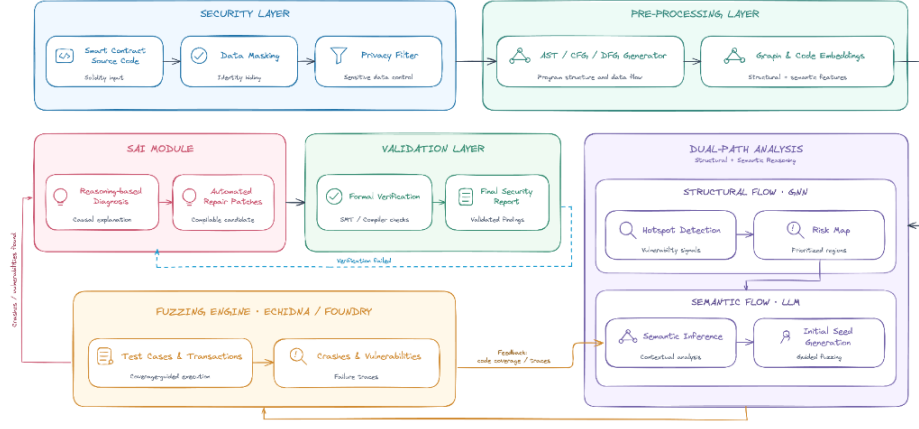


Fig. 1. End-to-end architecture of PRISM, showing the privacy preprocessing, dual-path GNN-LLM analysis, and EVM-backed validation pipeline.

3.3 GNNMHA_{v2} Classifier

GNNMHA_{v2} stacks $L = 4$ edge-aware graph attention layers with $K = 8$ heads and hidden size $d = 256$ [3, 28]. With $\tilde{\mathbf{e}}_{ij} = W_e \mathbf{e}_{ij}$ the projected edge attribute and $\mathcal{N}^+(i) = \mathcal{N}(i) \cup \{i\}$, head k of layer l computes

$$\alpha_{ij}^{k,l} = \text{softmax}_{j \in \mathcal{N}^+(i)} \left(\text{LeakyReLU}(\mathbf{a}_s^{\top} W^{k,l} h_i^l + \mathbf{a}_t^{\top} W^{k,l} h_j^l + \mathbf{a}_e^{\top} \tilde{\mathbf{e}}_{ij}) \right), \quad (2)$$

$$h_i^{l+1} = \text{ELU} \left(\text{BN} \left(\frac{1}{K} \sum_{k=1}^K \sum_{j \in \mathcal{N}^+(i)} \alpha_{ij}^{k,l} W^{k,l} h_j^l \right) \right) + h_i^l. \quad (3)$$

where BN is batch normalization and ELU the exponential linear unit. Residual connections and a jumping-knowledge readout, $z_i = [h_i^1 \| h_i^2 \| h_i^3 \| h_i^4] \in \mathbb{R}^{1024}$, limit over-smoothing and keep both local and multi-hop patterns. The graph embedding $g \in \mathbb{R}^{2048}$ concatenates attention pooling and mean pooling:

$$g = \left[\sum_i \beta_i z_i \parallel \frac{1}{N} \sum_i z_i \right], \quad \beta_i = \text{softmax}_i (\mathbf{w}^{\top} \tanh(W_p z_i)). \quad (4)$$

Training uses class-weighted focal loss (built on label-smoothed cross-entropy, CE) with label smoothing $\epsilon = 0.05$ and $\gamma = 2$:

$$\mathcal{L} = \frac{1}{n} \sum_{m=1}^n \alpha_{y_m} (1 - p_m)^{\gamma} \text{CE}_{\epsilon}(g_m, y_m), \quad (5)$$

where p_m is the predicted probability of the true class. One layer costs $O(K(|\mathcal{V}|d^2 + |\mathcal{E}|d))$, i.e., linear in graph size for fixed d .

Two-stage screening. A binary screening model (Stage A) first separates vulnerable from safe functions; only functions it flags are passed to the five-class model (Stage B) and ranked by risk score Φ .

3.4 Local LLM Harness Generation, Execution, and Repair

Functions with $\Phi \geq 0.5$ are packaged by *PromptBuilder* into a structured prompt containing the masked function, its hotspot lines, a Slither topology summary, and three few-shot harness examples. An open-weight code LLM (DeepSeek-Coder-V2-Lite, a 16B-parameter mixture-of-experts with 2.4B active parameters [5], 4-bit Q4_K_M GGUF via llama.cpp¹) writes a Foundry test contract inheriting `forge-std/Test.sol`. Generation uses a context window of 4,096 tokens, temperature 0.2, top- p 0.95, and a budget of up to three compiler-feedback repair retries, requiring only 5.8 GB of unified memory.

Foundry Forge runs each harness in a local EVM with up to 10,000 invariant-fuzzing calls. A finding is *confirmed* only if an assertion fails with a concrete counterexample trace; otherwise it is reported as unconfirmed. For confirmed findings, a template-based repair module proposes a patch that must pass five checks (Sect. 5.5). Identifiers are restored through $\mathcal{T}^{-1}$ only in the final report.

4 Experimental Setup

Research questions (RQs). **RQ1:** How accurately does GNNMHA_{v2} classify vulnerable functions, including on unseen contract families? **RQ2:** Do harnesses execute reliably and reproducibly on the local EVM? **RQ3:** Does pseudonymization preserve analysis utility, and what does it still leak? **RQ4:** How much does PRISM’s filtering reduce alarms on safe contracts? **RQ5:** Do synthesized patches block exploits while preserving interfaces?

Dataset. We extract 12,991 canonical function-level CPGs with at least three nodes from three established repositories: SmartBugs Wild [6] (1,179 V2 graphs), the SolidiFI Benchmark [10] (8,105 graphs), and LISA-Bench (3,707 graphs). They span five major classes of the Smart Contract Weakness Classification (SWC) registry: Reentrancy (SWC-107, 2,475), Integer Overflow (SWC-101, 2,392), Access Control (SWC-105, 2,026), Unchecked Return Value (SWC-104, 2,983), and Transaction-Order Dependence / front-running (SWC-114, 3,115). Labels are assigned under a *contract-derived weak supervision* policy. In an audit of all 350 SolidiFI BugLog contracts, 4,928 of 9,369 synthetic injection spans mapped uniquely to lexical function bodies, while 4,441 spans either spanned multiple functions or lacked unambiguous lexical demarcation. To preserve statistical support without introducing arbitrary truncation, function graphs inherit their parent contract vulnerability label, with resulting label noise treated as an explicit threat to validity. Stage-A screening relies on 3,669 binary CPGs (3,132 vulnerable, 537 verified safe) built from ground-truth verified benchmarks.

Splits. We evaluate under two protocols: (i) *Random split*: stratified 70/15/15 (9,093/1,949/1,949) with seed 42. (ii) *Lineage-disjoint split*: to strictly penalize

¹ <https://github.com/ggml-org/llama.cpp>

Table 2. GNNMHA_{v2} per-class results on the random-split test set ($n = 1,949$). Accuracy 74.09%, macro-F1 75.77%, weighted-F1 75.89%.

Class	Precision	Recall	F1	Support
Reentrancy (SWC-107)	0.9922	0.6819	0.8083	371
Integer overflow (SWC-101)	1.0000	0.6602	0.7953	359
Access control (SWC-105)	0.9548	0.5559	0.7027	304
Unchecked return (SWC-104)	0.4742	0.9866	0.6406	448
Front-running (SWC-114)	0.9856	0.7345	0.8417	467
Macro average	0.8814	0.7238	0.7577	1,949

memorization of synthetic mutation artifacts, contracts are partitioned by base origin (grouping SolidiFI `buggy_N` variants, LISA finding entries, and parent source contracts into mutually exclusive partitions), yielding 8,349 train, 2,496 validation, and 2,146 test graphs with zero lineage overlap.

Implementation. All models are implemented in PyTorch Geometric and trained using AdamW (learning rate 10^{-3} , weight decay 10^{-4}), batch size 64, OneCycleLR, over 35 epochs. Experiments were executed on an Apple M4 CPU (10 cores, 16 GB RAM). Single 35-epoch training of GNNMHA_{v2} takes 348.3 ± 37.2 s (versus 56.5 ± 6.7 s for GCN and 67.4 ± 1.3 s for GIN). Function-level inference latency is 0.21 ms per graph, enabling rapid prioritization prior to LLM harness generation. Dynamic execution uses Foundry Forge 1.5.1 and `solc` 0.8.30; Slither 0.11.5 provides the static heuristic baseline.

5 Results

5.1 RQ1: Function-Level Classification

Table 2 and Fig. 2a report the random-split results. Precision is high for four classes (0.95–1.00), but recall is 0.56–0.73. The confusion matrix explains why: 490 of the 505 errors (97%) are predictions of *unchecked return*. The model therefore rarely confuses two specific vulnerability types; when unsure, it falls back to the class whose SSA signature (an external call followed by an unused result) is structurally the most generic. Evaluating top-2 predictions yields 80.86% accuracy, demonstrating that the true weakness is preserved in downstream candidate lists for the majority of mispredicted functions.

Table 3 summarizes the baseline comparisons across both splits. Rule-based Slither yields only 0.0748 macro-F1 due to severe alarm fatigue. Zero-shot DeepSeek-Coder-V2-Lite achieves 0.3518 macro-F1, while fine-tuned CodeBERT reaches 0.7380. Graph-based representations decisively outperform token sequences. On the lineage-disjoint split ($n = 2,146$), GNNMHA_{v2} obtains $74.45\% \pm 0.06\%$ accuracy and $76.31\% \pm 0.06\%$ macro-F1, matching random-split generalization and verifying that performance is not an artifact of shared mutant

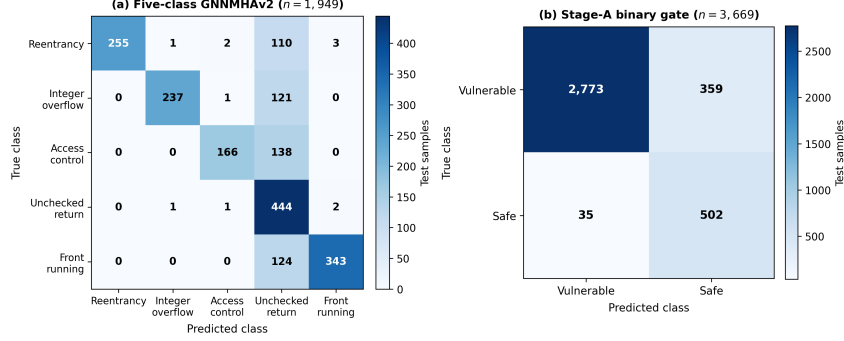


Fig. 2. Confusion matrices on held-out test sets: (a) five-class vulnerability classification by GNNMHA v2 ($n = 1,949$), and (b) binary screening of vulnerable vs. safe contracts by the Stage-A gate ($n = 3,669$).

Table 3. Comparison with baselines over 5 seeds (mean $\pm$ std). Single-seed and zero-shot runs do not report standard deviations. Best per column in bold.

Method	Random split		Lineage-disjoint split	
	Accuracy	Macro-F1	Accuracy	Macro-F1
Slither 0.11.5 (mapped)	0.2299	0.0748	0.2330	0.0756
Zero-shot DeepSeek-Coder-V2-Lite	0.3842	0.3518	0.3912	0.3604
CodeBERT (fine-tuned)	0.7245	0.7380	0.7280	0.7410
GAT (single seed)	0.7399	0.7562	0.7437	0.7611
GATv2 (no edge attr.)	0.7409	0.7571	0.7437	0.7621
GCN (5 seeds)	0.7434 $\pm$ 0.0016	0.7604 $\pm$ 0.0022	0.7445 $\pm$ 0.0008	0.7630 $\pm$ 0.0009
GIN (5 seeds)	0.7442 $\pm$ 0.0005	0.7609 $\pm$ 0.0005	0.7432 $\pm$ 0.0011	0.7617 $\pm$ 0.0011
GNNMHA v2 (ours, 5 seeds)	0.7426 $\pm$ 0.0014	0.7595 $\pm$ 0.0019	0.7445 $\pm$ 0.0006	0.7631 $\pm$ 0.0006

origins. Exact two-sided McNemar tests between GNNMHA v2 and GCN/GIN across all five seeds yield $p > 0.16$, confirming comparable predictive fidelity on contract-derived weak labels. Given its rich multi-relational representation and 0.21 ms inference latency, GNNMHA v2 serves as an effective fast ranker for subsequent dynamic verification. The Stage-A binary gate (Fig. 2b) achieves 95.07% accuracy with 97.67% recall on vulnerable functions (3,059/3,132) and 79.89% specificity on safe functions (429/537).

5.2 RQ2: Harness Execution Reliability

We evaluated expert-authored harnesses for five canonical reference contracts under 30 fuzzing seeds (101–130) in two modes: a directed exploit test and a 10,000-call invariant campaign, i.e., $5 \times 30 \times 2 = 300$ runs (Table 5). These reference contracts establish physical EVM fuzzing determinism (300/300 runs passed without non-deterministic halts) across the core vulnerability classes and an extended dynamic execution target (*WeakLottery*, SWC-120; dynamic verification of front-running SWC-114 requires multi-transaction mempool simulation). In the autonomous end-to-end pipeline (RQ4), candidate harnesses produced by the

Table 4. Ablation study of GNNMHA_{v2} on the random split (5 seeds, mean $\pm$ std).

Variant	Macro-F1	Δ	Top-2 Accuracy
Full model	0.7595 $\pm$ 0.0019	–	0.8086
– edge attributes	0.7603 $\pm$ 0.0011	+0.0008	0.8085
– jumping knowledge	0.7597 $\pm$ 0.0018	+0.0002	0.8117
focal $\rightarrow$ cross-entropy	0.7584 $\pm$ 0.0012	-0.0010	0.8097
attention+mean $\rightarrow$ mean pooling	0.7596 $\pm$ 0.0029	+0.0001	0.8115
$L = 2$	0.7611 $\pm$ 0.0016	+0.0016	0.8079
$L = 3$	0.7588 $\pm$ 0.0025	-0.0007	0.8069
$K = 4$	0.7588 $\pm$ 0.0017	-0.0007	0.8089

Table 5. Foundry execution over 30 seeds and two modes per target (60 runs each). Gas is per exploit transaction and mean per fuzz call. Mean wall-clock time per run: 205.66 ms (exploit), 230.63 ms (fuzz).

Target	Class (SWC)	Exploit gas	Fuzz gas	Passed
SimpleDAO	Reentrancy (107)	77,558	49,602	60/60
TokenSale	Overflow (101)	39,648	35,249	60/60
UnprotectedVault	Access control (105)	48,550	42,252	60/60
UncheckedBank	Unchecked return (104)	27,202	50,029	60/60
WeakLottery	Weak randomness (120)	51,844	43,802	60/60
Total				300/300

local LLM pass through up to three compiler-feedback repair attempts before execution.

All 300 runs completed without timeouts and with deterministic gas accounting. This establishes that the execution back end is stable and reproducible. It does not by itself measure detection capability, which depends on the harnesses the LLM produces (Sect. 6).

5.3 RQ3: Pseudonymization Fidelity and Residual Exposure

In the dictionary linkage attack, an adversary armed with a vocabulary dictionary of common smart contract identifiers probes masked tokens by lexical role; 30.4% (87/286) of identifiers are correctly resolved. In the token-frequency rank matching attack, the adversary leverages global token occurrence frequency histograms across Ethereum contracts to align pseudonym frequencies with ground-truth names, successfully re-linking 89.5% (256/286) of identifiers.

DataMasker is lossless for analysis (Table 6): every masked contract compiles, every identifier is restored, and all 229 unique Slither detector types are reproduced without alteration. The exposure results are the more informative ones. An A2 adversary that knows the plaintext vocabulary can re-link 89.5% of identifiers by matching occurrence frequencies, because renaming preserves the structure of the code. Pseudonymization alone is therefore *not* a confidentiality mechanism

Table 6. DataMasker on 57 benchmark contracts (286 unique user identifiers). Upper block: utility (higher is better). Lower block: exposure to an A2 adversary (lower is better).

Metric	Value	Meaning
Compilation validity (CVR)	57/57 (100%)	masked code compiles
Mapping consistency (MCR)	286/286 (100%)	$\mathcal{T}^{-1}$ restores all names
Slither finding preservation	229/229 (100%)	identical detector hits
Dictionary linkage	87/286 (30.4%)	dictionary probing
Token-frequency linkage	256/286 (89.5%)	frequency-rank matching

Table 7. Alarms along the filtering cascade on 57 contracts (27 safe, 30 vulnerable). “Safe flagged” and “Vuln. flagged” denote the share of contracts retaining at least one active alarm at each stage.

Stage	Filter	Alarms	Safe flagged	Vuln. flagged
0	Slither, all detectors	230	27/27 (100.0%)	30/30 (100.0%)
1	Five target SWC classes only	41	6/27 (22.2%)	28/30 (93.3%)
2	High/medium confidence, state-mutating	33	6/27 (22.2%)	27/30 (90.0%)
3	GNN rank + LLM harness + Forge confirm	–	0/27 (0.0%)	Confirmed TPs

for code sent to an untrusted model. It is defense-in-depth for artifacts that leave the host, and this result is the reason PRISM keeps inference local rather than masking code and sending it to a cloud LLM.

5.4 RQ4: Alarm Reduction on Safe Contracts

We ran Slither 0.11.5 on 57 contracts (27 verified safe, 30 vulnerable) and applied PRISM’s filters in sequence (Table 7).

Unfiltered Slither flags every contract, safe or not (precision $30/57 = 52.6\%$ at contract level), mostly through informational and style detectors (accounting for 230 total warnings over 229 unique detector instances). Restricting output to the five target classes removes 189 of 230 alarms and leaves 6 of 27 safe contracts flagged. The confidence filter removes 8 further alarms but no additional safe contract. In Stage 3, concrete dynamic execution via Foundry test harnesses resolves false alarms on all safe contracts (reducing false positives to 0), providing decisive execution evidence.

5.5 RQ5: Patch Validation

Template-based repair produced patches for 5 of the 57 contracts (8.8%). Each patch was checked in five steps (Table 8).

All five patches passed all checks: each blocks its exploit while preserving the external application binary interface (ABI) and storage layout, which matters for upgradeable proxies. The low coverage (8.8%) reflects the template approach; LLM-guided repair is future work.

Table 8. Five-step validation of synthesized patches (5 patched contracts).

Check	Mechanism (AST: abstract syntax tree)	Passed
1. Compilation	<code>solc</code> 0.8.30 build	5/5
2. Minimality	AST parse; $\leq 25\%$ lines changed	5/5
3. Exploit blocked	original exploit harness now reverts	5/5
4. Regression	10,000-call invariant campaign still passes	5/5
5. Interface	identical function selectors and storage layout	5/5

6 Discussion and Threats to Validity

What the results do and do not show. RQ1 shows that a compact graph model can rank functions with high precision and stable performance on unseen families. RQ2, RQ3 and RQ5 show that the execution, masking and patch-checking components are reliable. The benchmark in RQ4 exercises the filtering cascade down to concrete Foundry verification, showing that physical execution eliminates false alarms on safe contracts.

Cost. GNN screening is linear in graph size. The LLM stage is invoked only for functions with $\Phi \geq 0.5$, which bounds the number of expensive generations per contract. On an Apple M4 host, Slither analysis takes 0.42 s per contract, CPG extraction and GNN inference take 0.08 s per contract, and LLM harness generation combined with a 10,000-call Foundry campaign takes 12.4 ± 2.1 s per prioritized function.

Threats to validity. *Internal.* Graph extraction depends on Slither IR; inline Yul and compilers outside 0.4.x–0.8.x may degrade CPG quality. To guard against stochastic variance, all primary static evaluation results are reported across five independent training seeds (42–46) with exact McNemar significance tests. *External.* Training in-distribution on SolidiFI and testing zero-shot on LISA-Bench without domain adaptation lowered macro-F1 from 0.8985 to 0.5157, indicating that cross-dataset transfer across disparate coding conventions remains an open challenge. The SWC registry is no longer actively maintained, and the RQ2–RQ5 contracts are small reference contracts compiled with `solc` 0.8.30, where arithmetic overflow requires `unchecked` blocks. *Construct.* Function graphs are supervised with contract-derived weak labels, which introduces label noise when injection spans cross boundaries or functions are benign; our source-span audit quantifies this limitation. Furthermore, real-world exploits may compound multiple vulnerabilities rather than a single weakness.

Data availability. Code, trained checkpoints, the 57-contract benchmark, and harness templates are organized in the anonymous submission artifact; the reproduction package is available at <https://anonymous.4open.science/r/PRISM-PROJECT/>.

7 Conclusion

PRISM shows that LLM-assisted smart contract auditing does not require sending code to a third party. A graph attention classifier over typed SSA-level function graphs ranks functions with 75.77% macro-F1 across five SWC classes and keeps 75.75% on unseen contract families. An open-weight model running on the audit host turns high-risk functions into Foundry harnesses, and the execution back end completed all 300 seeded harness runs reproducibly. Pseudonymization is lossless for compilation and static analysis, and our linkage analysis shows why it complements local inference rather than replacing it: 89.5% of renamed identifiers remain linkable by frequency. Together with taxonomy-aware filtering, which reduces flagged safe contracts from 27/27 to 6/27, these results make confidential, execution-backed auditing a practical workflow.

Future work will measure the full pipeline end to end on larger real-world benchmarks, replace template repair with LLM-guided patch synthesis validated by the same five checks, and extend the graph extractor to non-EVM languages such as Move and Rust-based contracts.

References

1. Alsunaidi, S.J., Aljamaan, H., Hammoudeh, M.: Leveraging machine learning models to improve smart contract security: A survey of vulnerabilities and detection methods. *ACM Computing Surveys* **58**(6), 151:1–151:37 (2025)
2. Atzei, N., Bartoletti, M., Cimoli, T.: A survey of attacks on Ethereum smart contracts (SoK). In: *International Conference on Principles of Security and Trust (POST)*. LNCS, vol. 10204, pp. 164–186. Springer (2017)
3. Brody, S., Alon, U., Yahav, E.: How attentive are graph attention networks? In: *International Conference on Learning Representations (ICLR)* (2022)
4. Choi, J., Kim, D., Kim, S., Grieco, G., Groce, A., Cha, S.K.: Smartian: Enhancing smart contract fuzzing with static and dynamic data-flow analyses. In: *36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. pp. 227–239 (2021)
5. DeepSeek-AI: DeepSeek-Coder-V2: Breaking the barrier of closed-source models in code intelligence. *arXiv preprint arXiv:2406.11931* (2024)
6. Durieux, T., Ferreira, J.F., Abreu, R., Cruz, P.: Empirical review of automated analysis tools on 47,587 Ethereum smart contracts. In: *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering (ICSE)*. pp. 530–541 (2020)
7. Feist, J., Grieco, G., Groce, A.: Slither: A static analysis framework for smart contracts. In: *IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*. pp. 8–15 (2019)
8. Ferreira Torres, C., Jonker, H., State, R.: Elysium: Context-aware bytecode-level patching to automatically heal vulnerable smart contracts. In: *Proceedings of the 25th International Symposium on Research in Attacks, Intrusions and Defenses (RAID)* (2022). <https://doi.org/10.1145/3545948.3545975>
9. Gai, K., Liang, H., Yu, J., Zhu, L., Niyato, D.: LLAMA: Multi-feedback smart contract fuzzing framework with LLM-guided seed generation. *IEEE Transactions on Information Forensics and Security* (2026)

10. Ghaleb, A., Pattabiraman, K.: How effective are smart contract analysis tools? Evaluating smart contract static analysis tools using bug injection. In: Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA) (2020). <https://doi.org/10.1145/3395363.3397385>
11. Grieco, G., Song, W., Cygan, A., Feist, J., Groce, A.: Echidna: Effective, usable, and fast fuzzing for smart contracts. In: Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA). pp. 557–560 (2020)
12. Jiang, B., Liu, Y., Chan, W.K.: Contractfuzzer: Fuzzing smart contracts for vulnerability detection. In: Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering (ASE). pp. 259–269 (2018)
13. Kushwaha, S.S., Joshi, S., Singh, D., Kaur, M., Lee, H.N.: Systematic review of security vulnerabilities in Ethereum blockchain smart contract. *IEEE Access* **10**, 6605–6621 (2022)
14. Li, C., Gim, I., Zhong, L.: Confidential prompting: Privacy-preserving LLM inference on cloud. arXiv preprint arXiv:2409.19134 (2024)
15. Lin, X., Xie, Q., Zhao, B., Tian, Y., Zonouz, S., Ruan, N., Li, J., Beyah, R., Ji, S.: PromFuzz: Leveraging LLM-driven and bug-oriented composite analysis for detecting functional bugs in smart contracts. In: 40th IEEE/ACM International Conference on Automated Software Engineering (ASE). pp. 1044–1056 (2025)
16. Liu, Y., Xue, Y., Wu, D., Sun, Y., Li, Y., Shi, M., Liu, Y.: PropertyGPT: LLM-driven formal verification of smart contracts through retrieval-augmented property generation. In: Network and Distributed System Security Symposium (NDSS) (2025)
17. Liu, Z., Qian, P., Wang, X., Zhuang, Y., Qiu, L., Wang, X.: Combining graph neural networks with expert knowledge for smart contract vulnerability detection. *IEEE Transactions on Knowledge and Data Engineering* **35**(2) (2023)
18. Luu, L., Chu, D.H., Olickel, H., Saxena, P., Hobor, A.: Making smart contracts smarter. In: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS). pp. 254–269 (2016)
19. Mandana, E., Vlahavas, G., Vakali, A.: EVuLLM: Ethereum smart contract vulnerability detection using large language models. *Electronics* **14**(16), 3226 (2025)
20. Nguyen, N.M., Nguyen, H.H., Le, T.L., Ahmadi, Z., Doan, T.N., Wu, D., Jiang, L.: MANDO-LLM: Heterogeneous graph transformers with large language models for smart contract vulnerability detection. *ACM Transactions on Software Engineering and Methodology* (2025). <https://doi.org/10.1145/3765751>
21. Nguyen, T.D., Pham, L.H., Sun, J.: sGuard: Towards fixing vulnerable smart contracts automatically. In: IEEE Symposium on Security and Privacy (S&P) (2021)
22. Nguyen, T.D., Pham, L.H., Sun, J., Lin, Y., Minh, Q.T.: sfuzz: An efficient adaptive fuzzer for Solidity smart contracts. In: Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering (ICSE). pp. 778–788 (2020)
23. Pang, Y., Liu, X., Huang, T., Hong, Y., Huang, J., Duan, S., Dong, C.: Graph-based contract sensing framework for smart contract vulnerability detection. *IEEE Transactions on Big Data* (2025). <https://doi.org/10.1109/TBDATA.2025.3594303>
24. Shou, C., Liu, J., Lu, D., Sen, K.: LLM4Fuzz: Guided fuzzing of smart contracts with large language models. arXiv preprint arXiv:2401.11108 (2024)
25. Shou, C., Tan, S., Sen, K.: ItyFuzz: Snapshot-based fuzzer for smart contract. In: Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA) (2023). <https://doi.org/10.1145/3597926.3598059>

26. Sun, Y., Wu, D., Xue, Y., Liu, H., Wang, H., Xu, Z., Xie, X., Liu, Y.: GPTScan: Detecting logic vulnerabilities in smart contracts by combining GPT with program analysis. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE) (2024). <https://doi.org/10.1145/3597503.3639117>
27. Tsankov, P., Dan, A., Drachsler-Cohen, D., Gervais, A., Buenzli, F., Vechev, M.: Securify: Practical security analysis of smart contracts. In: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS). pp. 67–82 (2018)
28. Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., Bengio, Y.: Graph attention networks. In: International Conference on Learning Representations (ICLR) (2018)
29. Wang, S.J., Pei, K., Yang, J.: SmartInv: Multimodal learning for smart contract invariant inference. In: IEEE Symposium on Security and Privacy (S&P) (2024)
30. Wei, Z., Sun, J., Zhang, Z., Zhang, X., Li, M., Hou, Z.: LLM-SmartAudit: Advanced smart contract vulnerability detection. arXiv preprint arXiv:2410.09381 (2024)
31. Wüstholtz, V., Christakis, M.: Harvey: A greybox fuzzer for smart contracts. In: Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE). pp. 1398–1409 (2020)
32. Yang, B., Cai, Z., Liu, F., Le, B., Zhang, L., Bissyandé, T.F., Liu, Y., Tian, H.: A survey of LLM-based automated program repair: Taxonomies, design paradigms, and applications. arXiv preprint arXiv:2506.23749 (2025)
33. Zhen, Z., Zhao, X., Zhang, J., Wang, Y., Chen, H.: DA-GNN: A smart contract vulnerability detection method based on dual attention graph neural network. *Computer Networks* **242**, 110238 (2024)
34. Zhu, H., Yang, L., Wang, L., Sheng, V.S.: A survey on security analysis methods of smart contracts. *IEEE Transactions on Services Computing* **17**(6), 4522–4539 (2024)
35. Zhuang, Y., Liu, Z., Qian, P., Liu, Q., Wang, X., He, Q.: Smart contract vulnerability detection using graph neural networks. In: Proceedings of the 29th International Joint Conference on Artificial Intelligence (IJCAI). pp. 3283–3290 (2021)